In addition to adding the view, you'll also need to register it. Add a `path` object defining `"restricted_page/"` to RiffMates/bands/urls.py. With the new view and route in place, load up the development server and visit http://localhost:8000/bands/restricted _page/.

Did it work? Depending on what state you were in when you visited the page, you may have seen the expected message or an error. If you were still authenticated as the superuser, from using the Django Admin, you would be able to see the page. Any authenticated account sees You Are Logged In on the restricted page.

If you got an error message, go back to the Django Admin (http://localhost:8000/admin/), log in, and then visit the page. To see the error message, go to the Django Admin, and in the top-right corner, click the Log Out link. When you click the link, the Django Admin informs you that you are successfully logged out. Once logged out, go back to the restricted page. This time, you will see a `404` message.

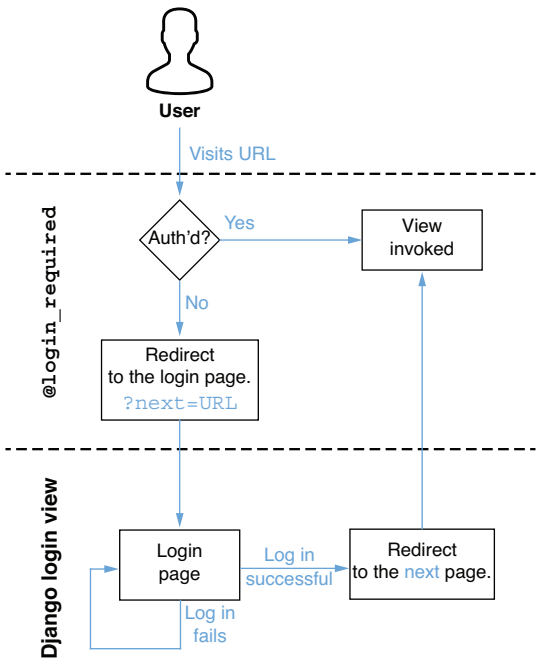


Figure 6.4 Authentication verification steps for visiting a `@login_required` wrapped view

The `404` error is not generated because you aren't authenticated; it is because there is no login screen yet. The `@login_required` decorator checks your session to see if you are authenticated, and if you are not, it redirects to the login page. Django has a default value for the login page of /accounts/login/. If you look closely at the `404` message, you'll see this URL along with an extra query parameter called `next`. The value of `next` is where the login view will send you if you successfully authenticate. Figure 6.4 shows the steps involved in visiting a `@login_required` wrapped view.

Django treats the login for Django Admin as a special case and does its own authentication. This is because you may or may not use the Django Admin in your site, and your site's login page should look like your site, not the Django admin. Django provides a built-in view for authentication, but you have to write the template to go with it.

There are several views Django provides for authentication and user management; they are all in the `django.contrib.auth.urls` module and can be included directly into your site by adding them to your main urls.py file. Add the following to